

Lecture 07b.

Trees :
Traversals and Array-based
representations

Tree Traversals

- Given a data structure, a common task is to traverse all stored data elements
 - visit each element precisely once
 - visit in some systematic and meaningful order
 - Note “visit” means “process the contents” but does not include just “passing through using the links”
- For an array, or linked list, the natural way is a simple forward scan
 - For trees there are more natural options

Traversals: “Visiting” vs “via”

- In traversing a tree we will be following the edges between the nodes
- We distinguish
 - “Via” merely passing through a node and not looking or processing its stored contents
 - “Visit”/“process”. Handle or use actual contents, e.g. print them out.
 - That is: “visit” means “process the contents” but just “passing through using the links” does not count as “process”
- For an array, or linked list, the natural way is a simple forward scan
 - For trees we will see some natural options:

Ordering of Parent and Children

- For each set of “parent and children” we need to make a primary decision of which one(s) we process first.
- Two immediate choices:
 - “**Preorder**”: Parent first then children
 - “**Postorder**”: Children first, then Parent afterwards
- Mnemonic: “P” of “**P**re or **P**ost” matches “P” of “**P**arent”
 - so **P**re/**P**ost tells us the position of the **P**arent

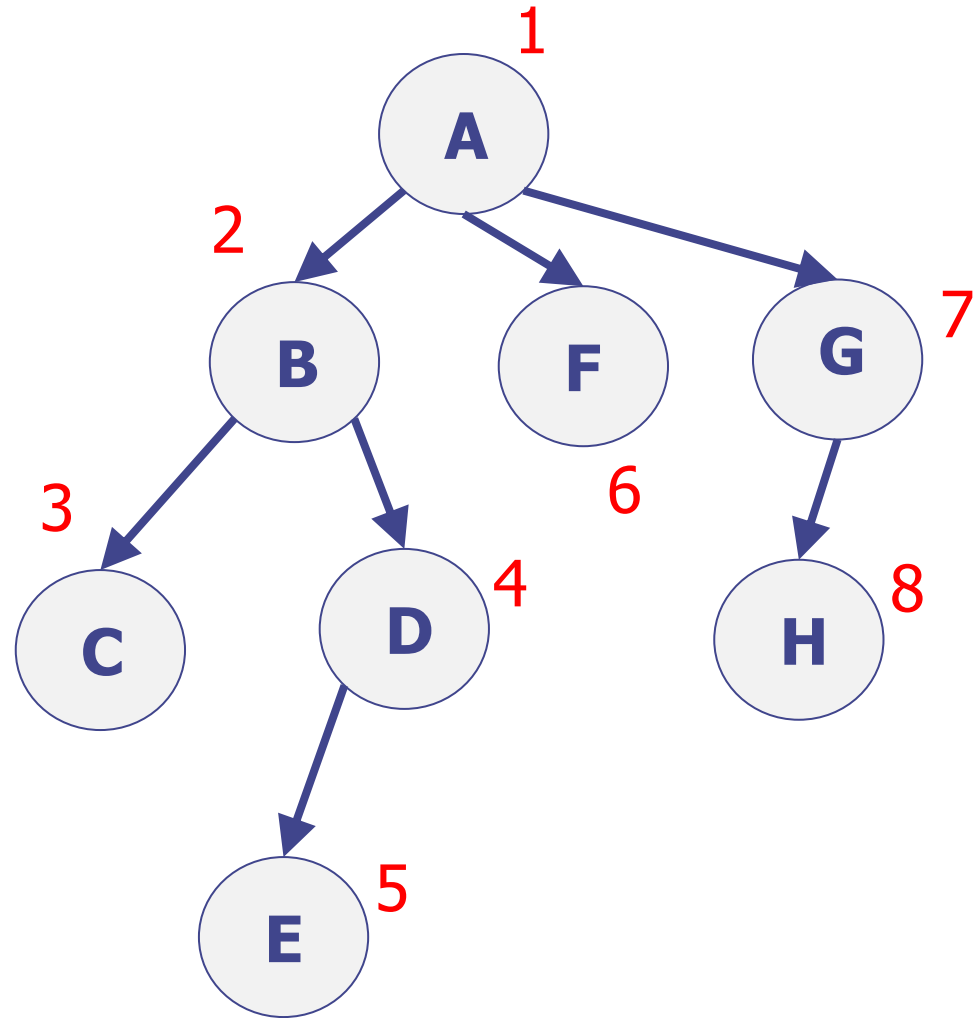
Traversal: “**Preorder**”

- For each set of “parent and children” we process the parent first.

```
Preorder(Node n) {  
    process(n)  
    foreach child c of n  
        Preorder(c)  
}
```

Traversal: Preorder example

- Preorder(A) processes A and then calls
Preorder(B)
Preorder(F)
Preorder(G)
- Preorder(B) processes B then calls
Preorder(C) giving "C"
Preorder(D) giving "DE"
- Etc.
- Putting this together we get the traversal order
A(BCDE)F(GH)
or
ABCDEFGH



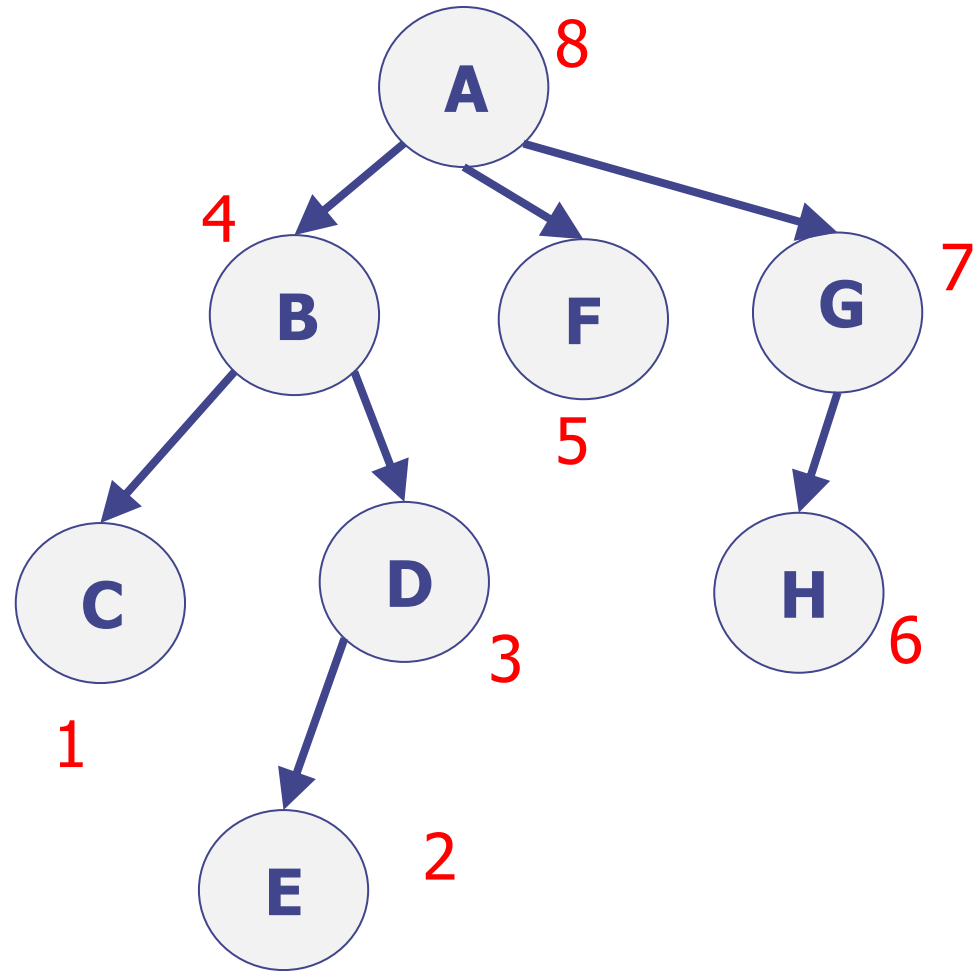
Traversal: “**Postorder**”

- For each set of “parent and children” we process the parent afterwards.

```
Postorder(Node n) {  
    foreach Child c of n  
        Postorder(c)  
    process(n)  
}
```

Traversal: Postorder example

- Preorder(A) calls
Preorder(B)
Preorder(F)
Preorder(G)
process(A)
- Preorder(B) calls
Preorder(C) giving "C"
Preorder(D) giving "ED"
process(B)
- Etc.
- Putting this together we get
the traversal order
(C(ED)B)F(HG)A
or
CEDBFHGA



Binary Trees: Parent vs Children

- In a binary tree, there are (at most) two children, “left” and “right”
- So, another natural option is to put the parent in the middle
 - “Inorder” : left child, then Parent, then right child
 - (Of course, one could do right-parent-left, but tends to be the same, so ignore it here.)
 - Mnemonic?: In the usual representation it looks like a horizontal scan following the order from left to right
- The Inorder traversal is very important later for “Binary Search Trees”

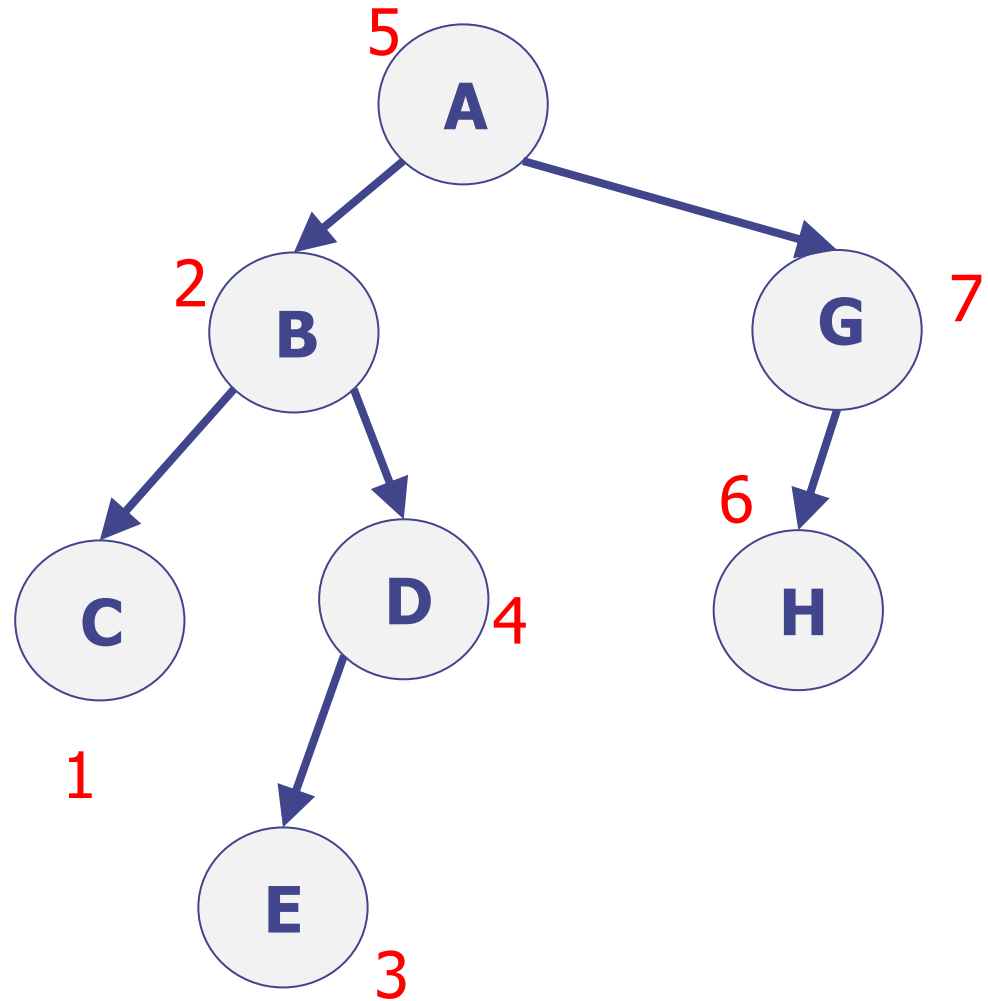
Traversal: "Inorder"

- Follow: left-parent-right:

```
Inorder(Node n) {  
    Inorder(n.left)  
    process(n)  
    Inorder(n.right)  
}
```

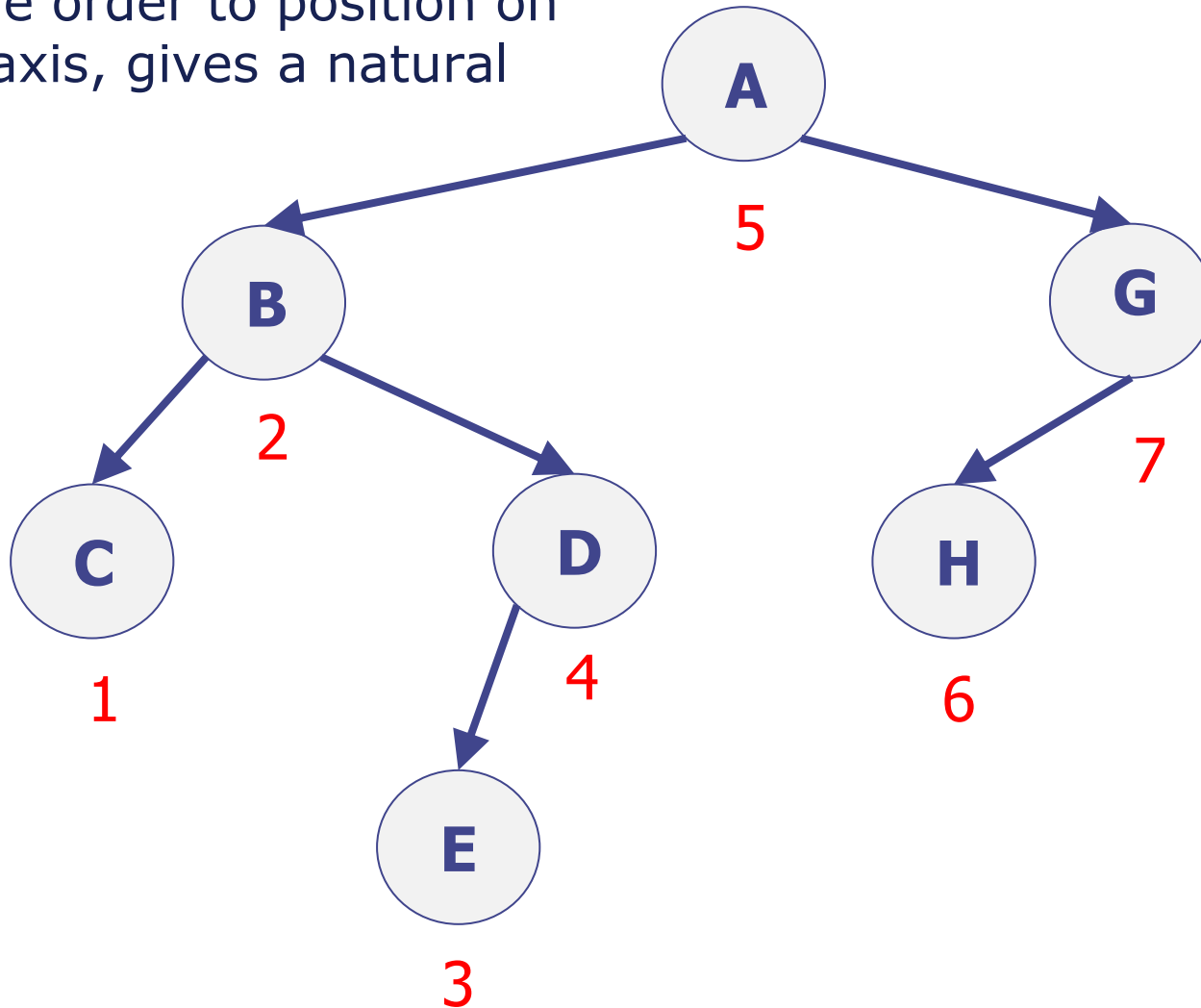
Traversal: Inorder example

- From root call
Inorder(B)
process(A)
Inorder(G)
- Inorder(B) calls
Inorder(C) giving "C"
process(B) giving "B"
Inorder(D) giving "ED"
- Inorder(G) gives "HG"
- Putting this together we get
CBEDAHG



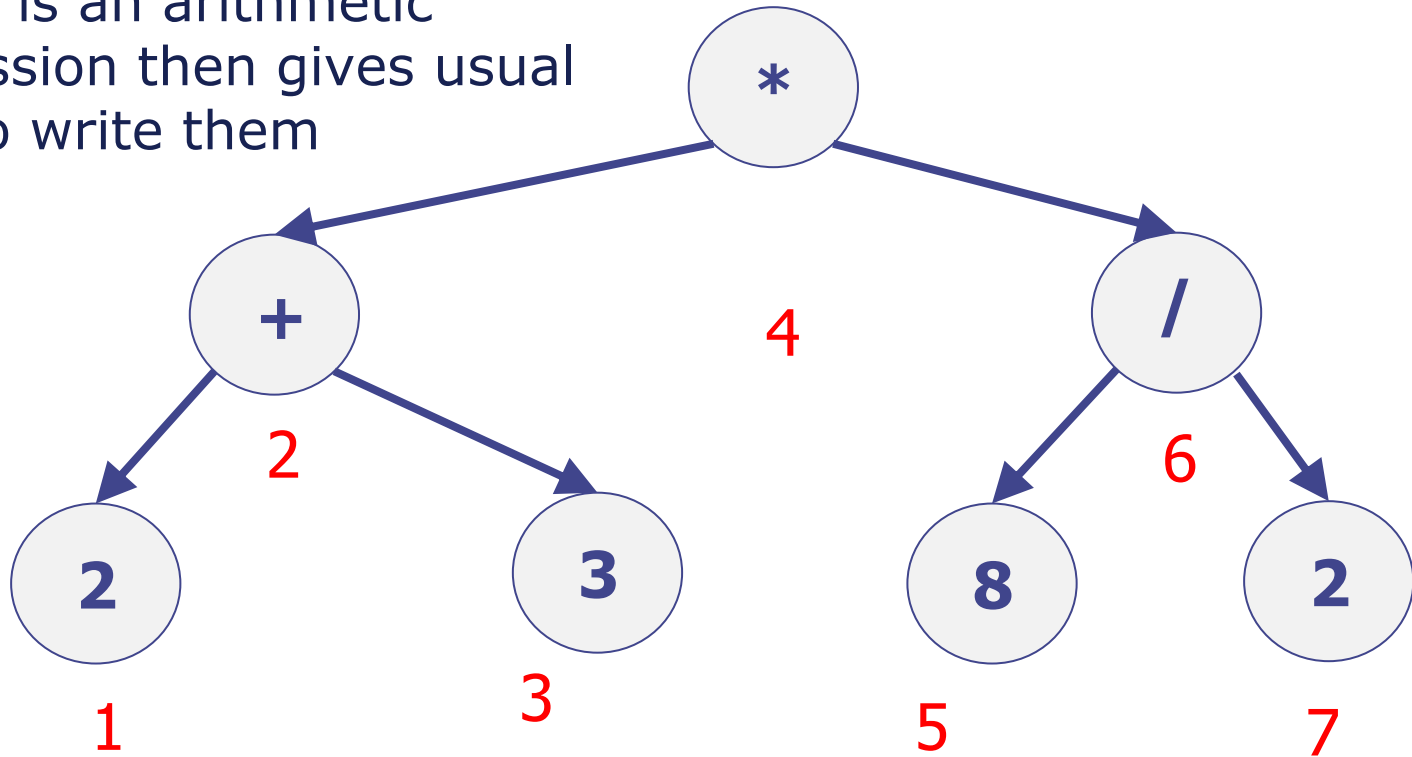
Traversal: Inorder example (2)

- Use the order to position on the x axis, gives a natural layout



Traversal: Inorder example (3)

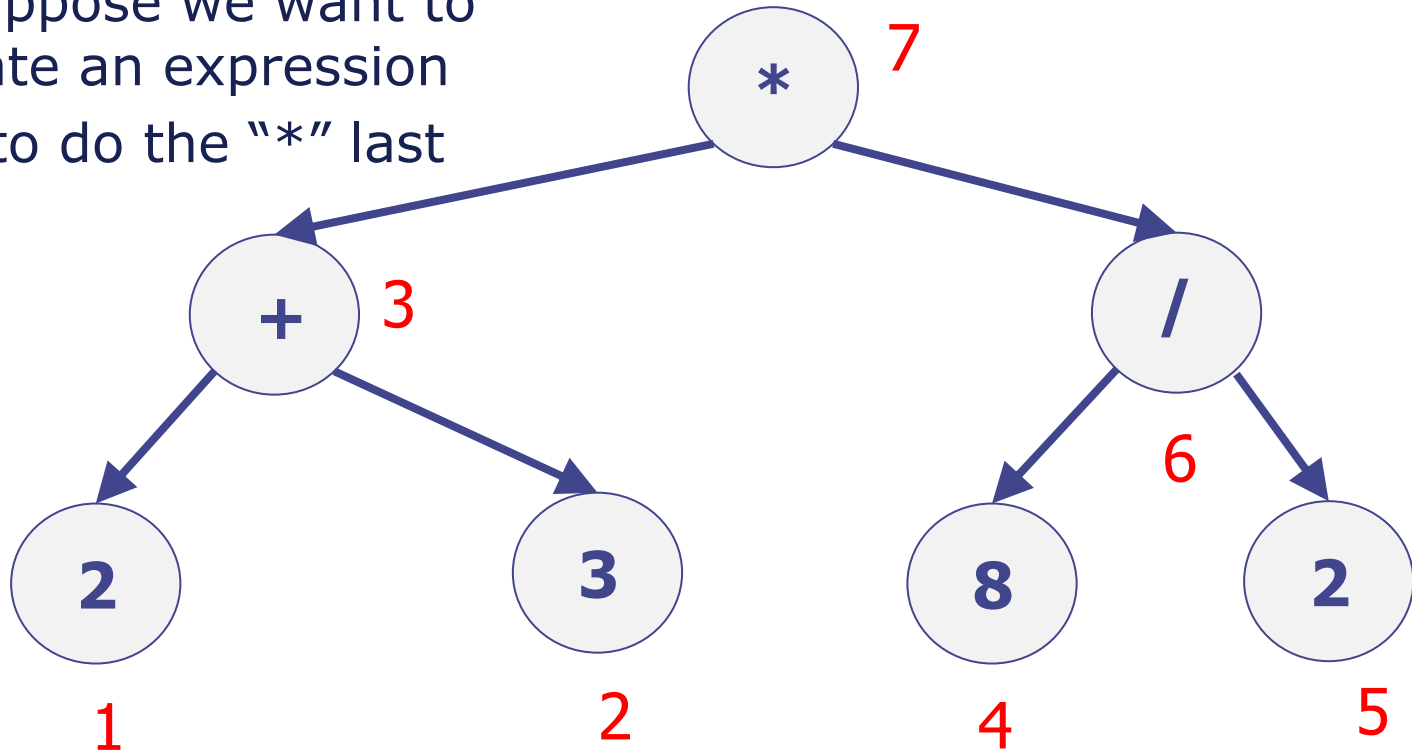
- If tree is an arithmetic expression then gives usual way to write them



- Would print as
 $(2 + 3) * (8 / 2)$

Traversal: Postorder example (3)

- But suppose we want to evaluate an expression
- Need to do the "*" last



Use postorder traversal :
evaluate the children subtrees
before process the parent
operator

Reprise of Representations

- We have seen “linked structures”
 - These are natural given how we usually draw trees
- But are there alternatives?
 - And why might we care?

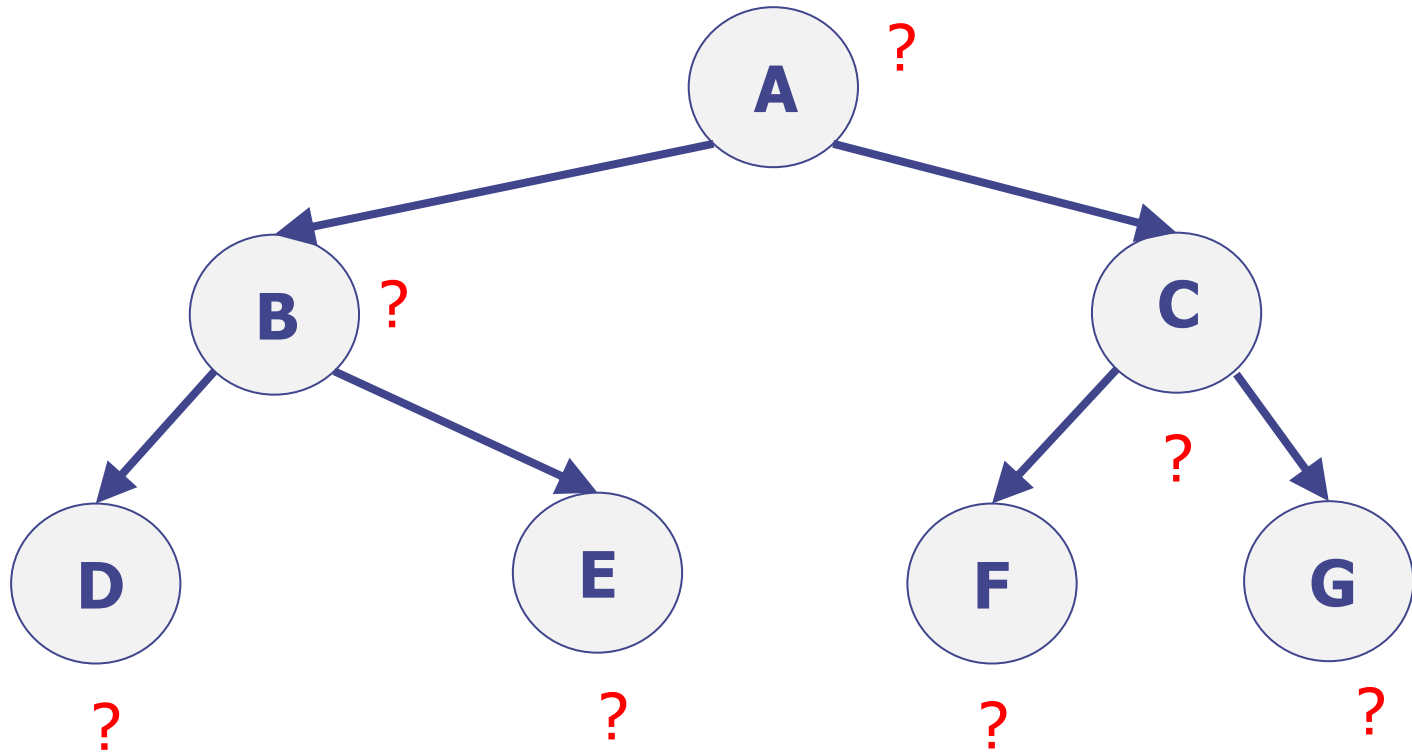
Arrays vs. Linked Lists

- Arrays: Fast and compact
 - Pro: No extra pointers
 - Pro: Good memory locality
 - fewer cache misses
 - Con: Awkward to do insertions/deletions
- Linked Lists: Bigger but flexible
 - Pro: Easier to do insertions/deletions
 - Need to modify some pointers, but most nodes do not move in the memory
 - Con: More space used
 - Con: Poor memory locality - More cache misses
- Ideally want to get the benefit of both

Improving Tree Representations?

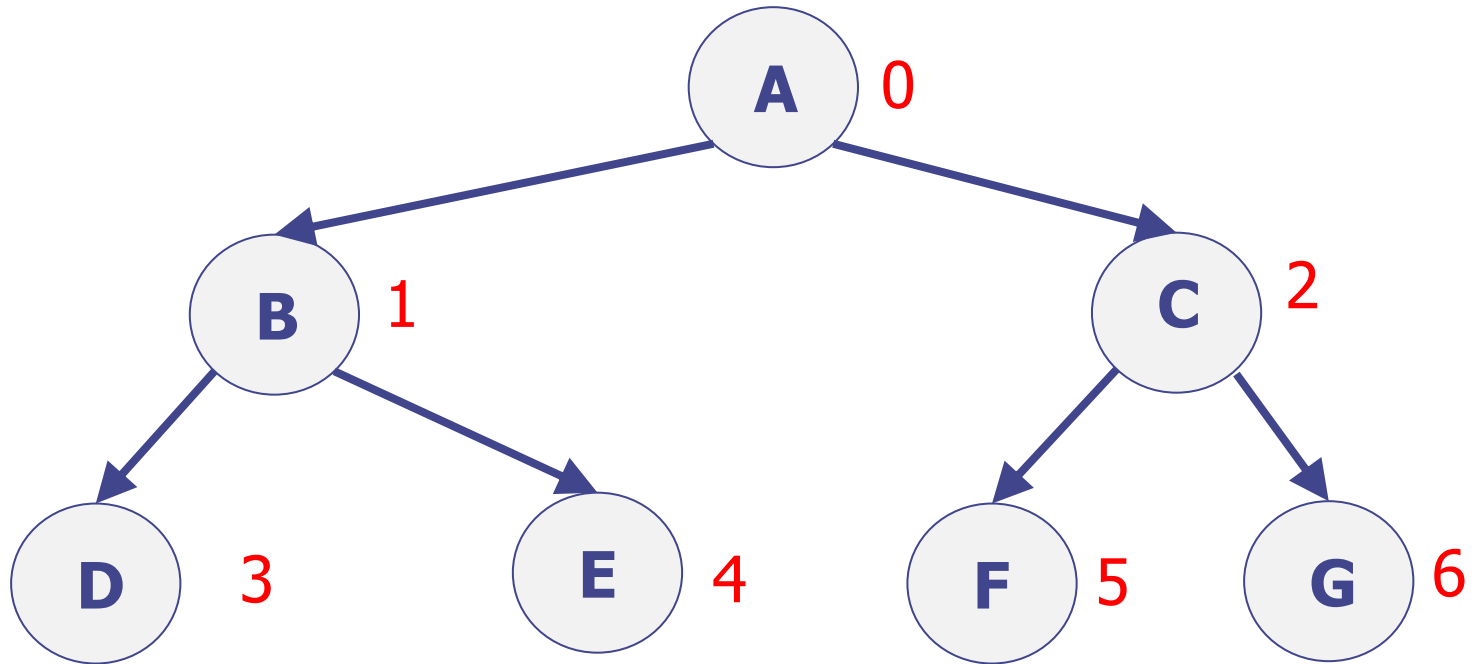
- Can we consider storing a tree inside an array?
 - ADT: behaves like a tree at the user level. Needed so as to match tree operations
 - CDT: implemented using an Array. Good for memory locality and avoiding pointers
- Might seem confusing, but is a nice example of “OO-style” of “data encapsulation”

Tree to Array: How?



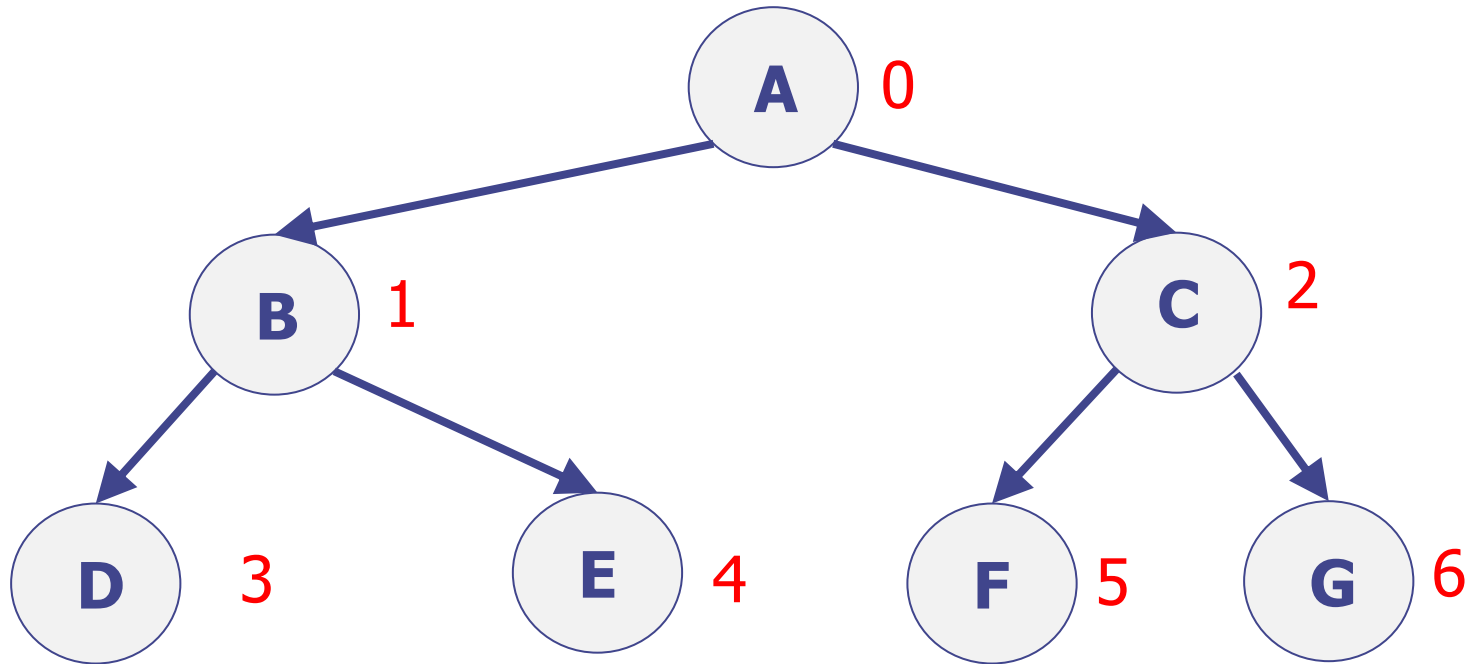
- We have to associate every node with some unique index in an array, the “?”
- There are many ways we might do this!
- How do we decide?

Tree to Array Mapping: Criteria



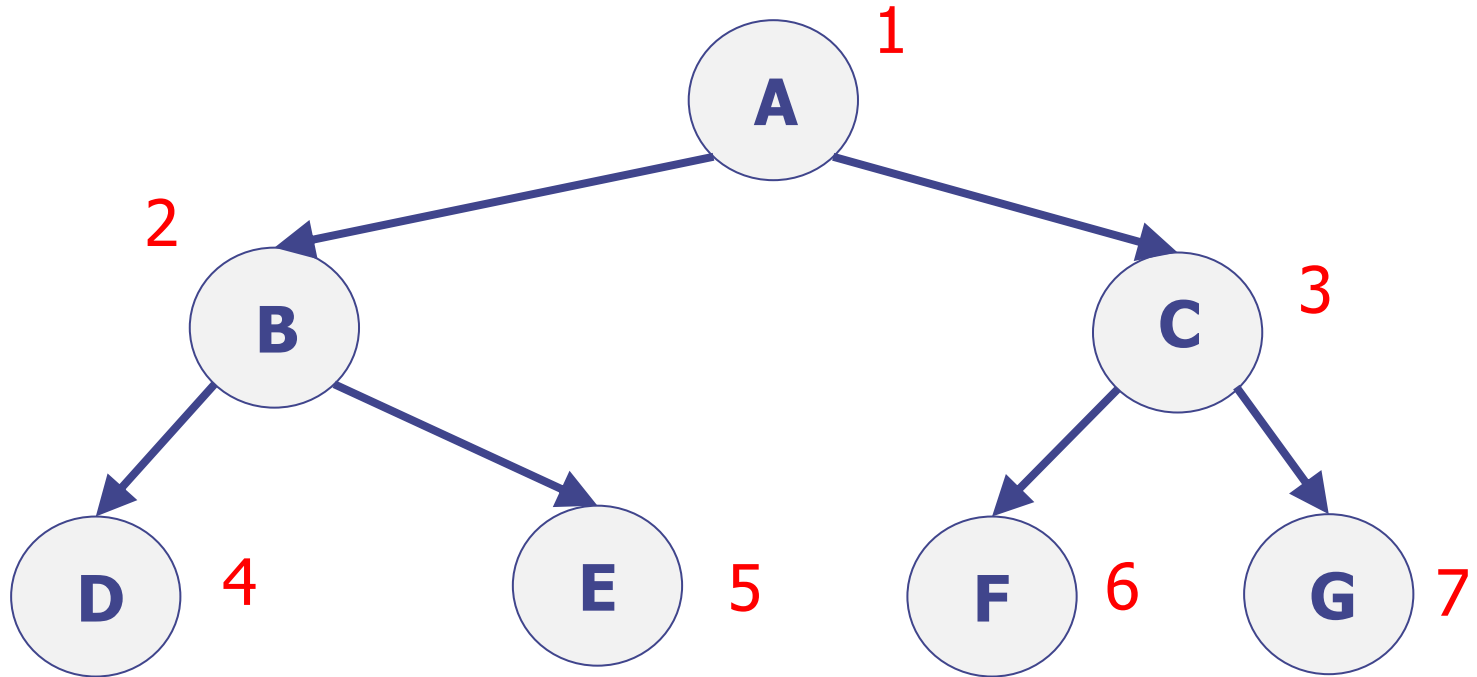
- Generally, might be reasonable to have shallower nodes earlier in the tree.
- Might as well put "left" earlier than "right"
- Given that arrays start at index=0, the above seems reasonable?

Tree to Array Mapping: Criteria (2)



- Also, need a clear easy mapping between indices of parents and children
 - E.g. map from parent 0 to children 1 or 2
 - E.g. move child 4 back to parent 1
- There is a mapping but is a bit messy (Exercise offline)

Tree to Array Mapping: Standard



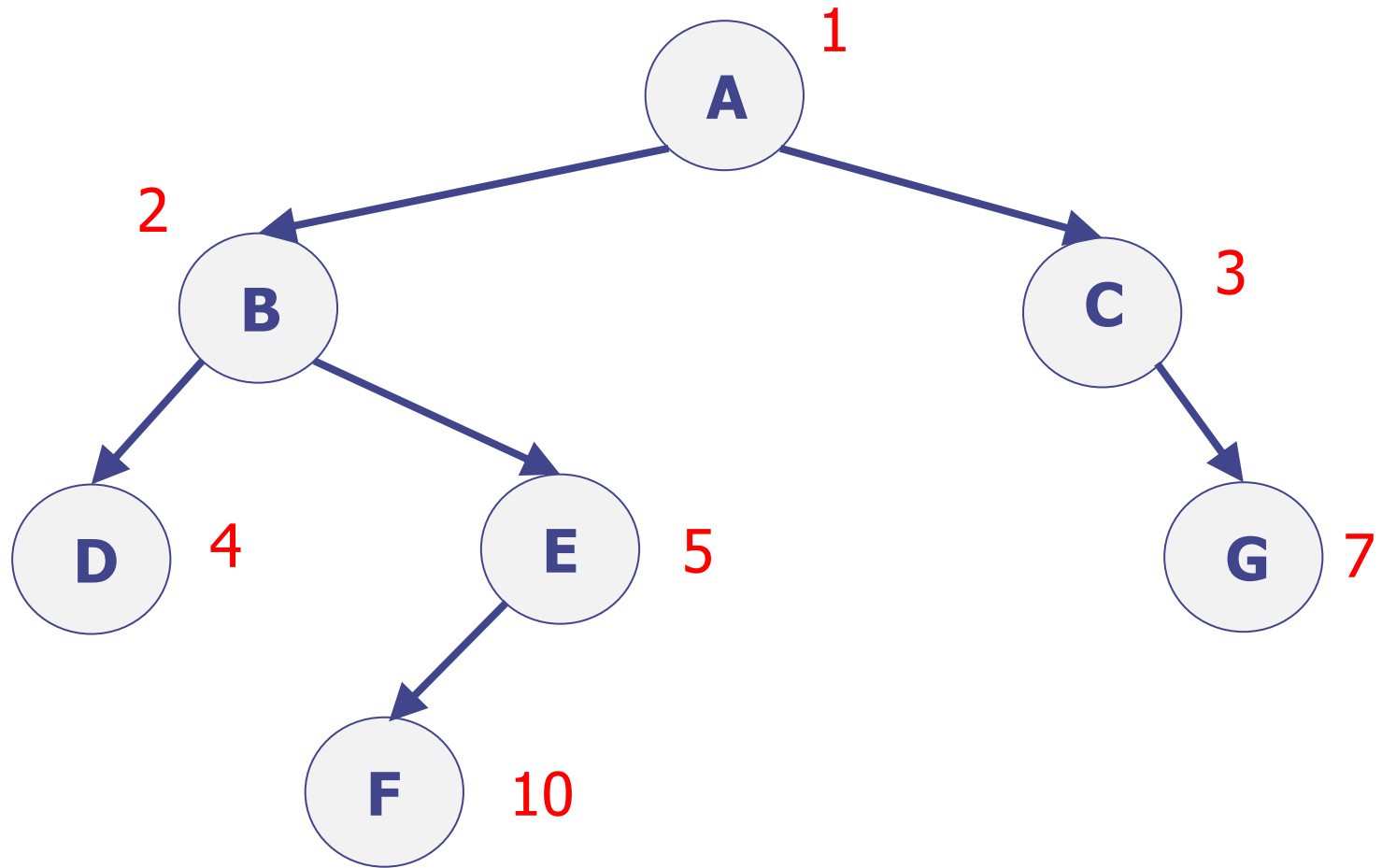
- Better, more convenient, mapping is given above
 - E.g. map from parent 2 to left child 4 is “multiply by 2”
 - E.g. move child 5 back to parent 2 is “integer divide by 2”
- Index=0 is not used
 - very slight waste of space, but well worth it in terms of ease of use

Tree to Array: Standard Map

Call the index of a node n its "rank" $r(n)$

- Then:
 - For the root: $r(\text{root}) = 1$
 - For a given parent node p at $r(p)$:
 - Place the left child at $2 * r(p)$
 - Place the right child at $2 * r(p) + 1$
- If nodes are missing we leave a blank entry, we will use "-"

Binary Tree to Array Mapping:



Gives

0	1	2	3	4	5	6	7	8	9	10
-	A	B	C	D	E	-	G	-	-	F

Implementation

- Remember that if think of the rank, $r(n)$ of node n , as a binary number (index) then
 - “times 2” is a left shift “ $<<1$ ”
- $r(n) = r(\text{par}(n)) << 1 + 0$ for left
- $r(n) = r(\text{par}(n)) << 1 + 1$ for right
- E.g. $r(\text{par}(n)) = 101$ gives children at
 - “101”+“0” = 1010
 - “101”+“1” = 1011
- Going to the parent is a right shift
- Hence, implementations of this can be very fast – used in binary heaps later.

Exercise (offline)

- Why is this representation correct?
 - I.e. does the mapping between array satisfy needed uniqueness properties?
 - Is it true that each element of the array corresponds to a unique node of the tree?
 - E.g. If I claim that it is incorrect, then how would you convince me otherwise?
- Hint: from the previous slide think of the rank written as binary number
 - Realise that it describes the “L” vs. “R” decisions on going from the root.
 - Relate to each number having a unique binary representation

Size of the array?

When we get to properties of trees, we will see that

- A tree of height h will need an array size of (about) 2^h
- This is potentially exponential in the number of nodes in the tree
 - If the tree is far from complete – such as a chain, then most of the array will be unused

Misc. Comments

- Advantages of the tree-as-array structure:
 - Saves space as do not have to store the pointers – they are replaced by fast computations
 - The storage can be more compact – “better memory locality” and this can be good because of cache and memory hierarchies – when an array element is accessed then other entries can be pulled into the cache, and so access becomes faster.
- Con: If the tree is not close to being complete then it can require large array and many entries will be unused

Next sub-lecture Lec07c

- Properties of trees
- Connections between
 - Numbers of nodes, n
 - Height, h